

Verification of Lorentz covariance:

Lorentz transformation

vs

Liénard–Wiechert potential

학번 : 20241292

이름 : 박은재

목차

1 목표

2 물리 개념

Liénard–Wiechert 포텐셜

3 코드 설명

4 실행 결과

5 결론 및 계획

목표

로렌츠 공변성을 로렌츠 변환과 LW(=Liénard-Wiechert) 포텐셜을 이용하여 검증

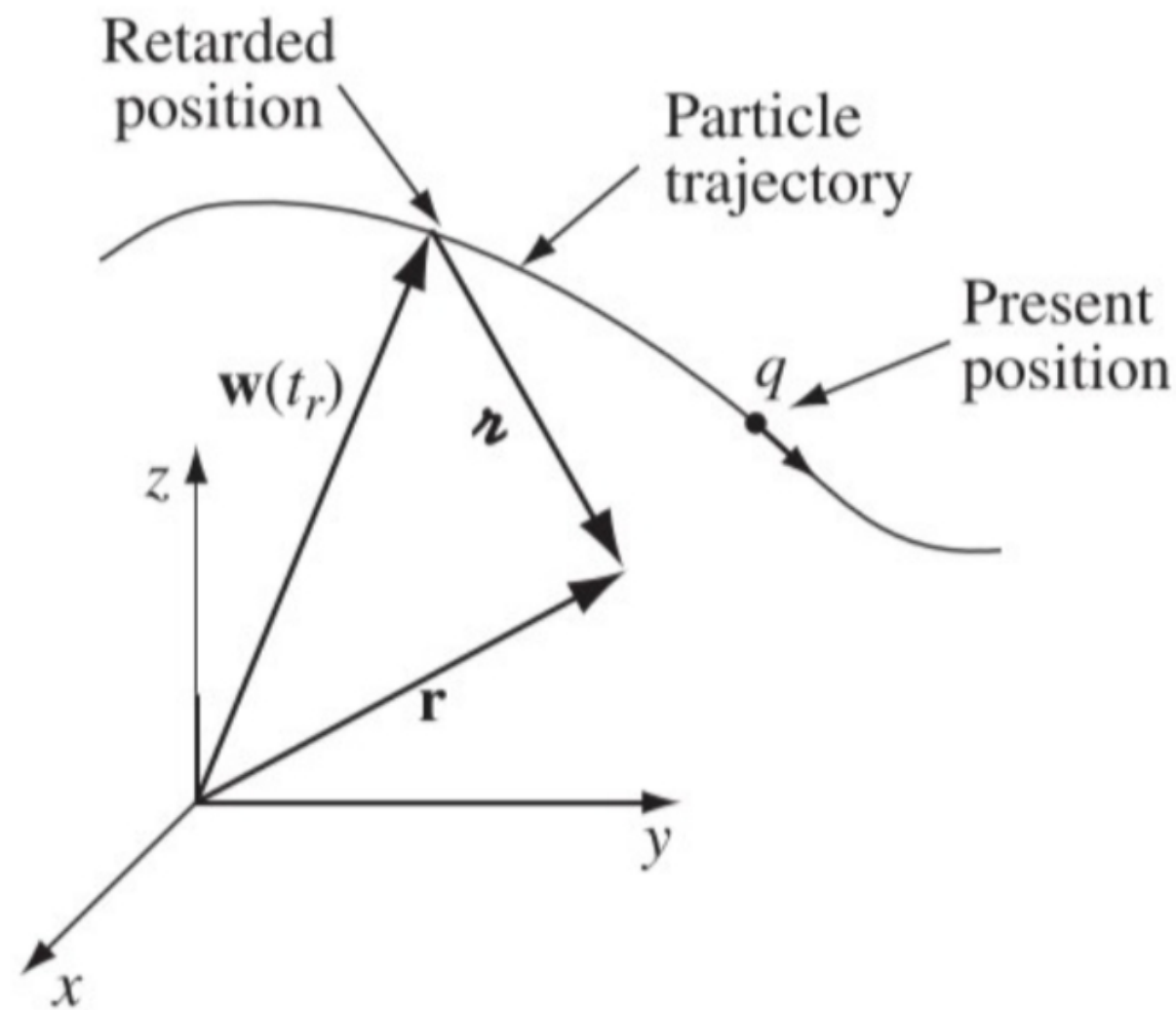
- n 개 점전하로 인해 발생하는 전기장 격자 점 각각에서 계산

- 로렌츠 변환을 통해 위의 전기장을 v 의 속도로 움직이면서 관측하는 관찰자의 시점에서의 필드 계산
- LW 포텐셜을 통해 전하들이 $-v$ 의 속도로 움직일 때 관찰자가 관측하는 필드 계산

- 두 방법을 통해 각각 계산한 필드를 비교하여 오차 분석 → 로렌츠 공변성 검증
- 병렬화의 효율성 측정

물리적 배경

Liénard-Wiechert potential : 전하가 과거(지연된) 위치에서 만든 전기·자기 퍼텐셜을 계산해 그로부터 전자기장을 구함



$$\vec{r}' = \vec{r} - \vec{r}', \quad t_r = t - \frac{r'}{c}$$

$$\vec{r}_{\text{LW}} = \vec{r} - \vec{w}(t_r^{\text{LW}}), \quad t_r^{\text{LW}} = t - \frac{r_{\text{LW}}}{c}$$

$$\vec{E} = \frac{q}{4\pi\epsilon_0} \frac{r_{\text{LW}}}{(\vec{r}_{\text{LW}} \cdot \vec{u})^3} \left[(c^2 - v^2) \vec{u} + \vec{r}_{\text{LW}} \times (\vec{u} \times \vec{a}) \right]$$

$$\vec{B} = \frac{1}{c} \hat{r}_{\text{LW}} \times \vec{E} \quad (\vec{u} \equiv c\hat{r}_{\text{LW}} - \vec{v})$$

코드 설명

ChargeField

TensorField

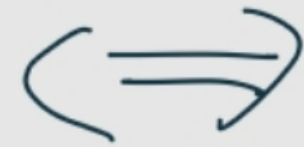
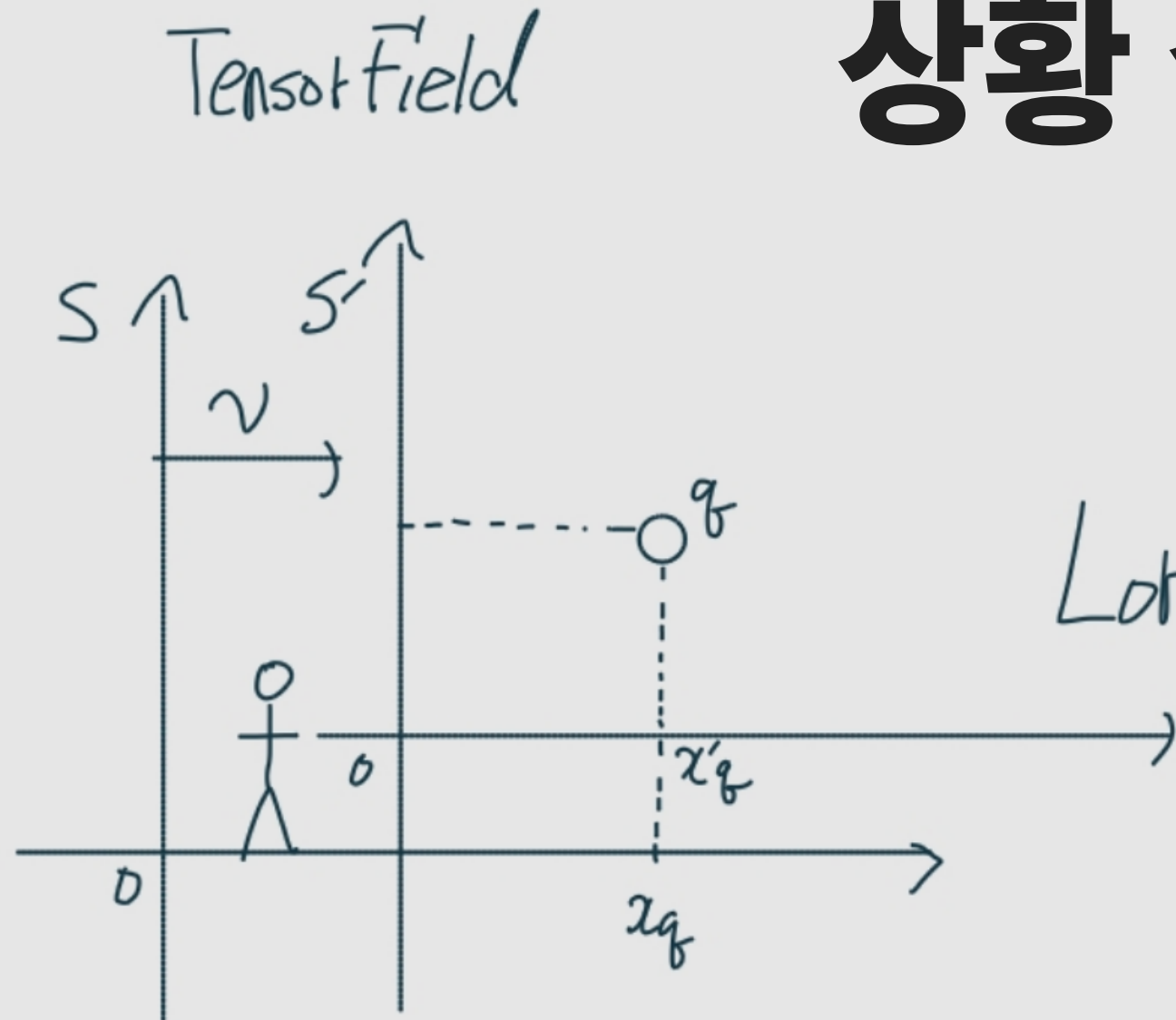
LWField

FieldVisualizer

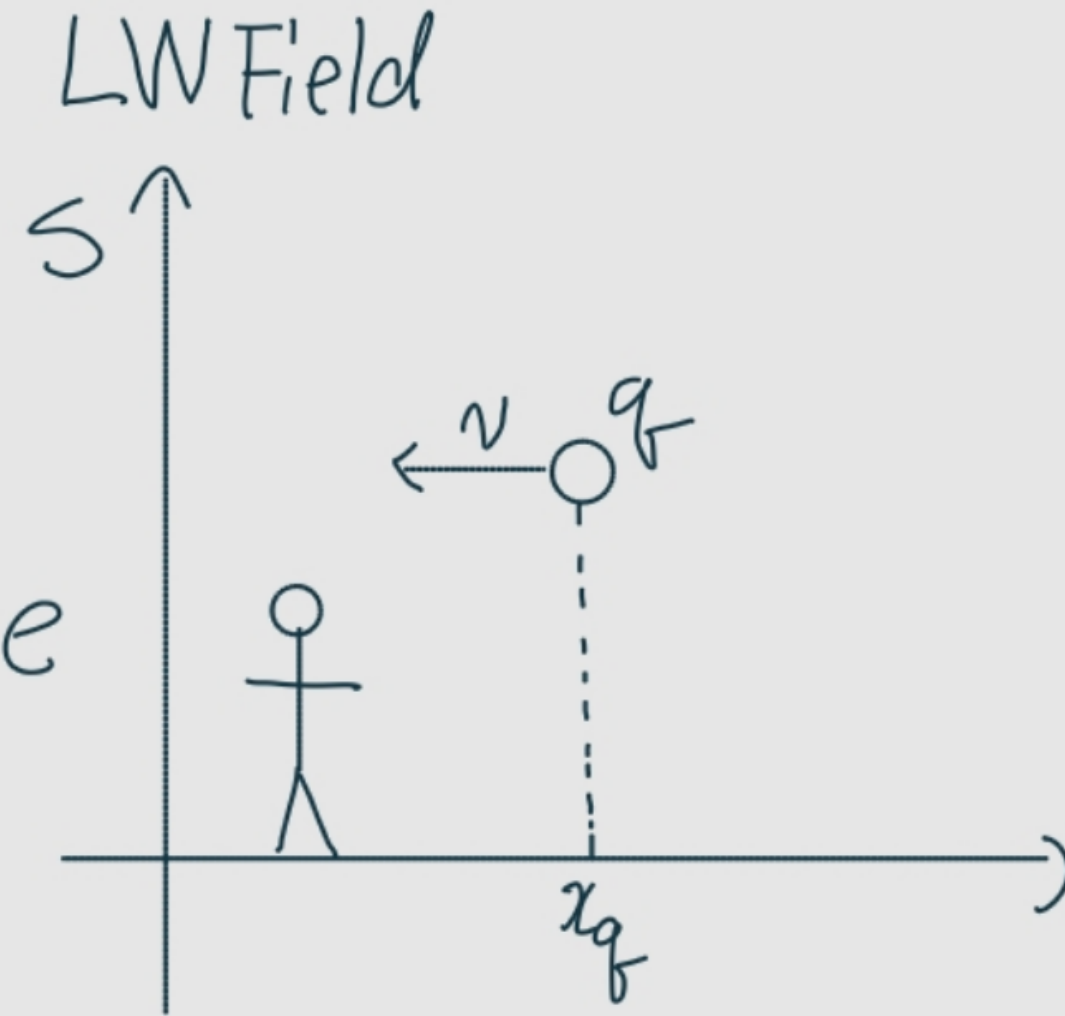
CovarianceVerifier

ParallelBenchmark

상황 설명



Lorentz Covariance



- 관찰자는 각 그리드마다 존재함.
- 둘다 S에서의 값을 반환함.

x_q 입력됨

$$\rightarrow x'_q = L^{-1} x_q \rightarrow F'(x') \text{ 계산}$$

$$\rightarrow L F'(x') L^T = F(x)$$

(x 는 원점에서의 Δx 의 축약
 S 와 S' 의 원점 $(0,0,0,0)$ 이 일치할 때를 기준)

ChargeField

- 전하 add, random하게 생성, 되돌리기, 지우기, 전하 배치 저장 및 불러오기 등 전하 배치와 관련된 기능
- 변환되지 않은 static field 계산

```
@dataclass
class PointCharge:
    q: float          # 전하량
    pos: np.ndarray   # 위치 벡터
```

```
class ChargeField:
    def __init__(self, shared_charges=None):
        if shared_charges is not None:
            self.charges = shared_charges
        else:
            self.charges = []
```

ChargeField

```
@staticmethod #병렬화는 self 있으면 못씀
def _calculate(X, Y, Z, target_charges):
    Ex = np.zeros_like(X)
    Ey = np.zeros_like(Y)
    Ez = np.zeros_like(Z)
    singularity_mask = np.zeros_like(X, dtype=bool)

    for charge in target_charges:
        rx = X - charge.pos[0]
        ry = Y - charge.pos[1]
        rz = Z - charge.pos[2]
        R_sq = rx**2 + ry**2 + rz**2

        # 특이점(전하 위치) 처리
        too_close = R_sq < 1e-10
        singularity_mask |= too_close
        R_sq[too_close] = 1.0
        R_cube = R_sq ** 1.5

        Ex += k*charge.q * rx / R_cube
        Ey += k*charge.q * ry / R_cube
        Ez += k*charge.q * rz / R_cube

    # 특이점 nan으로 처리해서 그래프로 그리지 않도록 함.
    Ex[singularity_mask]=np.nan
    Ey[singularity_mask]=np.nan
    Ez[singularity_mask]=np.nan

    return Ex, Ey, Ez
```

$$\vec{E} = k \frac{q}{|\vec{r}|^2} \frac{\vec{r}}{|\vec{r}|} = k \frac{q}{|\vec{r}|^3} \vec{r}$$

ChargeField

```
def calc_static_field_parallel(self, X, Y, Z, override_charges=None, num_cores=None):
    target_charges = override_charges if override_charges is not None else self.charges

    if num_cores is None: num_cores=cpu_count()

    chunks_X = np.array_split(X, num_cores)
    chunks_Y = np.array_split(Y, num_cores)
    chunks_Z = np.array_split(Z, num_cores)

    tasks = []
    for i in range(num_cores):
        tasks.append((chunks_X[i], chunks_Y[i], chunks_Z[i], target_charges))
    with Pool(num_cores) as pool:
        results = pool.starmap(ChargeField._calculate, tasks)

    # 4. 조립
    Ex = np.concatenate([res[0] for res in results])
    Ey = np.concatenate([res[1] for res in results])
    Ez = np.concatenate([res[2] for res in results])

    return Ex.reshape(X.shape), Ey.reshape(Y.shape), Ez.reshape(Z.shape)
```

병렬화
multiprocessing의
Pool 이용
각 격자점을 나눠서 계산.

TensorField

- 로렌츠 변환으로 v 로 이동하는 관찰자가 보는 필드를 계산
 - 수축되기 이전의 필드 계산 → 로렌츠 변환으로 관찰자가 본 필드 반환

```
def _transf_system(self, X,Y,Z, charges, velocity):  
    ...  
    좌표와 전하를 정지계 기준의 좌표로 변환  
    ...  
    L=Lorentzmatrix(-np.array(velocity))  
  
    #좌표 변환  
    T=np.zeros_like(X) #시간 축 벡터 생성  
    Coords=np.stack([T, X, Y, Z], axis=0)  
  
    Coords_prime=np.einsum('ij,j...->i...',L,Coords)  
  
    X_rest, Y_rest, Z_rest = Coords_prime[1], Coords_prime[2], Coords_prime[3]  
  
    #전하 변환  
    transformed_charges = []  
    for charge in charges:  
        q_vec = np.array([0, charge.pos[0], charge.pos[1], charge.pos[2]])  
  
        q_vec_prime = L @ q_vec  
  
        p_rest = q_vec_prime[1:4]  
        transformed_charges.append(PointCharge(charge.q, p_rest))
```

$$x' = \Lambda^{-1}x$$

TensorField

```
def get_transformed_field(self, X, Y, Z, velocity):
    total_workload = X.size * len(self.charges)
    X_rest, Y_rest, Z_rest, charges_rest = self._transf_system(X, Y, Z, self.charges, velocity)

    if total_workload < 1_000_000:
        Ex, Ey, Ez = self.calc_static_field_parallel(X_rest, Y_rest, Z_rest, override_charges=charges_rest)
    else:
        Ex, Ey, Ez = self.calc_static_field(X_rest, Y_rest, Z_rest, override_charges=charges_rest)

    L = Lorentzmatrix(velocity)

    F = np.zeros(Ex.shape + (4, 4))

    #E_i=c*F_0i
    F[..., 0, 1] = EX / c; F[..., 1, 0] = -EX / c
    F[..., 0, 2] = Ey / c; F[..., 2, 0] = -Ey / c
    F[..., 0, 3] = EZ / c; F[..., 3, 0] = -EZ / c

    F_prime = L @ F @ L #L==L^T

    return self.get_prime_fields(F_prime)
```

$$x' = \Lambda^{-1}x$$

F'(x') 계산

$$F(x) = L F'(x') L^T$$

LWField

- Liénard–Wiechert 방법을 사용하여 -v로 움직이는 전하를 관측한 필드 반환

```
@staticmethod
def _get_charge_pos(q_r0,v,a,t):
    return q_r0+v * t + 0.5 * a * t**2 #등가속도 운동으로 가정
```

```
@staticmethod
def _retarded_eq(tr, t,r0,q_r0,v0,a0):
    ...
    shoot function
    f(tr)=c*(t-tr)-|r-w(tr)|을 반환(0과의 오차)
    해당 함수의 root가 지연시간 tr
    ...
```

```
w_tr=LWField._get_charge_pos(q_r0,v0,a0,tr)
r_LW = r0- w_tr
r_LW_mag = np.linalg.norm(r_LW)
return c * (t - tr) - r_LW_mag
```

$$\vec{r}_{\text{LW}} = \vec{r} - \vec{w}(t_r^{\text{LW}})$$

$$t_r^{\text{LW}} = t - \frac{r_{\text{LW}}}{c}$$

```
@staticmethod
def _solve_tr(r0,q_r0, v0,a0,t=0):
    t_max=t
    t_min=t-10*RANGE_LIMIT/c
    try:
        tr=optimize.brentq(LWField._retarded_eq, t_min, t_max, args=(t,r0,q_r0,v0,a0),xtol=1e-12)
    except ValueError:
        return np.nan
    return tr
```

Shooting Algorithm 참고

LWField

```
@staticmethod
def _calculate(flat_X, flat_Y, flat_Z, velocity, charges):
    d=0
    #검증을 위한 속도, 가속도 설정
    v=-np.array(velocity)
    a=np.zeros(3) #검증을 위한 등속도 상황이므로 일단 0으로 둬
```

```
Grid_length = len(flat_X)
```

```
E_flat=np.zeros((Grid_length,3))
B_flat=np.zeros((Grid_length,3))
```

```
for charge in charges:
    for i in range(Grid_length):
        r0=np.array([flat_X[i], flat_Y[i], flat_Z[i]])
        tr=LWField._solve_tr(r0,charge.pos, v, a)
        if np.isnan(tr):
            d+=1
            continue
```

```
#field 계산에 필요한 요소 계산
w_tr=LWField._get_charge_pos(charge.pos,v,a,tr)
r_LW = r0 - w_tr
r_LW_mag = np.linalg.norm(r_LW)
if r_LW_mag < 1e-10: #singular point 처리
    E_flat[i]=np.nan
    B_flat[i]=np.nan
    continue
r_n=r_LW / r_LW_mag
u=c * (r_LW / r_LW_mag) - v
v_sq=np.sum(v**2)
```

```
E_vec= k*charge.q* r_LW_mag * (((c**2)-v_sq)*u + np.cross(r_LW, np.cross(u, a)))/(np.dot(r_LW,u)**3)
```

```
B_vec= np.cross(r_n, E_vec)/c
E_flat[i]+=E_vec
B_flat[i]+=B_vec
```

```
return E_flat, B_flat
```

$$(\vec{u} \equiv c\hat{r}_{\text{LW}} - \vec{v})$$

$$\vec{E} = \frac{q}{4\pi\epsilon_0} \frac{r_{\text{LW}}}{(\vec{r}_{\text{LW}} \cdot \vec{u})^3} \left[(c^2 - v^2)\vec{u} + \vec{r}_{\text{LW}} \times (\vec{u} \times \vec{a}) \right]$$

$$\vec{B} = \frac{1}{c} \hat{r}_{\text{LW}} \times \vec{E}$$

LWField

병렬화

```
def calc_field_parallel(self, x,y,z, velocity, num_cores=None):
    flat_X = x.flatten()
    flat_Y = y.flatten()
    flat_Z = z.flatten()
    if num_cores is None: num_cores=cpu_count()
    chunks_X = np.array_split(flat_X, num_cores)
    chunks_Y = np.array_split(flat_Y, num_cores)
    chunks_Z = np.array_split(flat_Z, num_cores)

    tasks = [(chunks_X[i], chunks_Y[i], chunks_Z[i], velocity, self.charges) for i in range(num_cores)]

    with Pool(num_cores) as pool:
        results = pool.starmap(self._calculate, tasks)

    E_flat = np.concatenate([r[0] for r in results])
    B_flat = np.concatenate([r[1] for r in results])
    E_prime = E_flat.reshape(X.shape + (3,))
    B_prime = B_flat.reshape(X.shape + (3,))
    E_out = np.moveaxis(E_prime, -1, 0)
    B_out = np.moveaxis(B_prime, -1, 0)
    return E_out,B_out
```

FieldVisualizer

- TensorField와 LWField에서 계산된 전자기장을 plotly를 이용하여 3차원에서 벡터 형태로 표현하고 상호작용 가능하게 함.

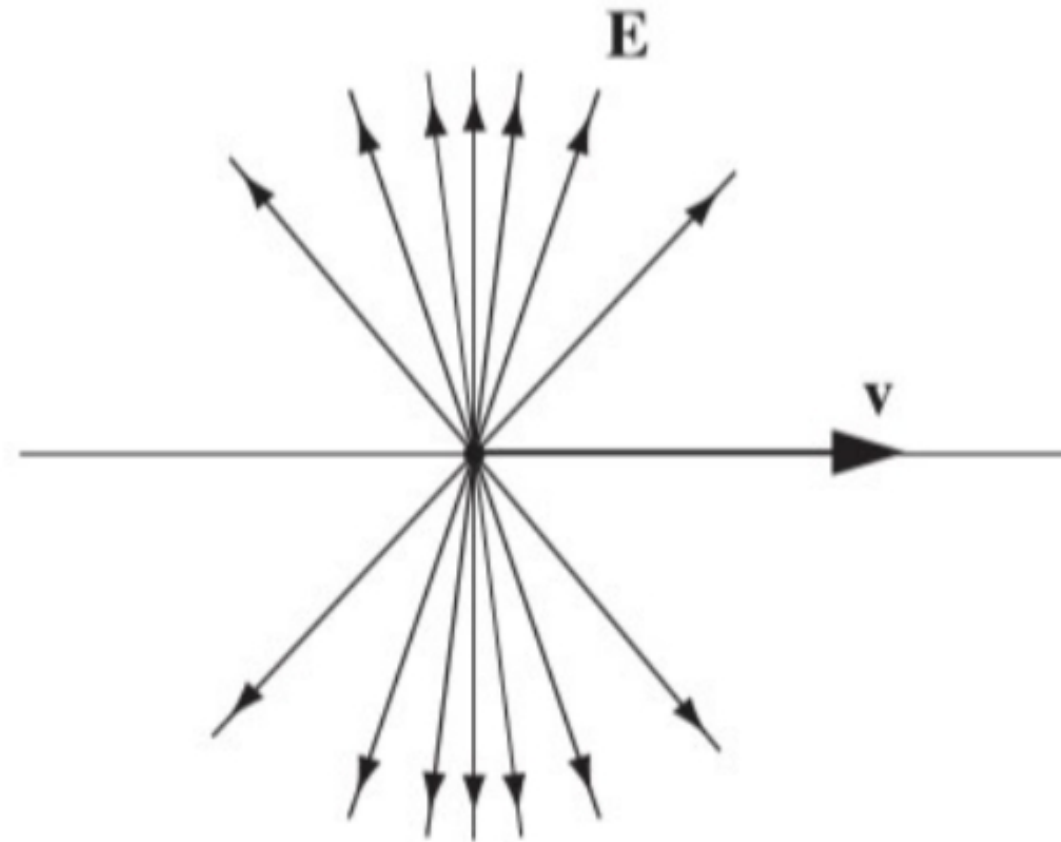
CovarianceVerifier

- 속도에 따라서 TensorField와 LWField에서 계산된 전자기장의 오차를 계산하고 그래프를 그림.

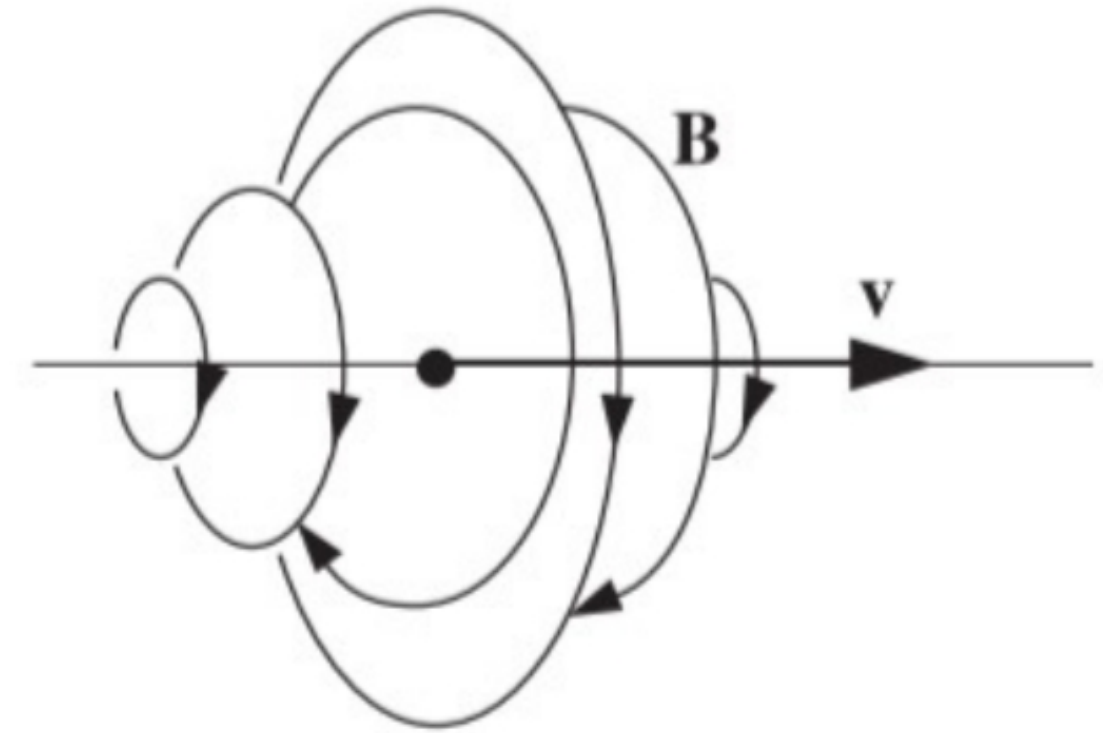
ParallelBenchmark

- 일정한 작업량을 코어수를 늘려가며 시간 측정
- 최대 코어수에서 작업량을 늘려가며 시간 측정

실행 결과

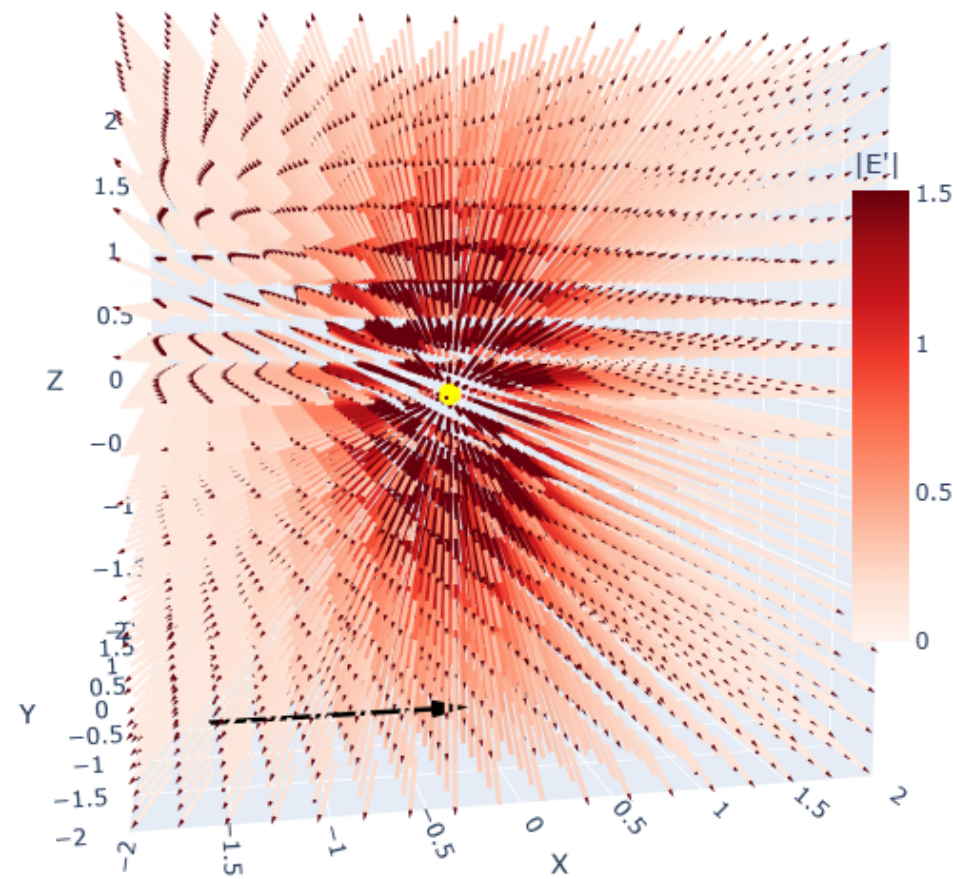


Relativistic EM Field

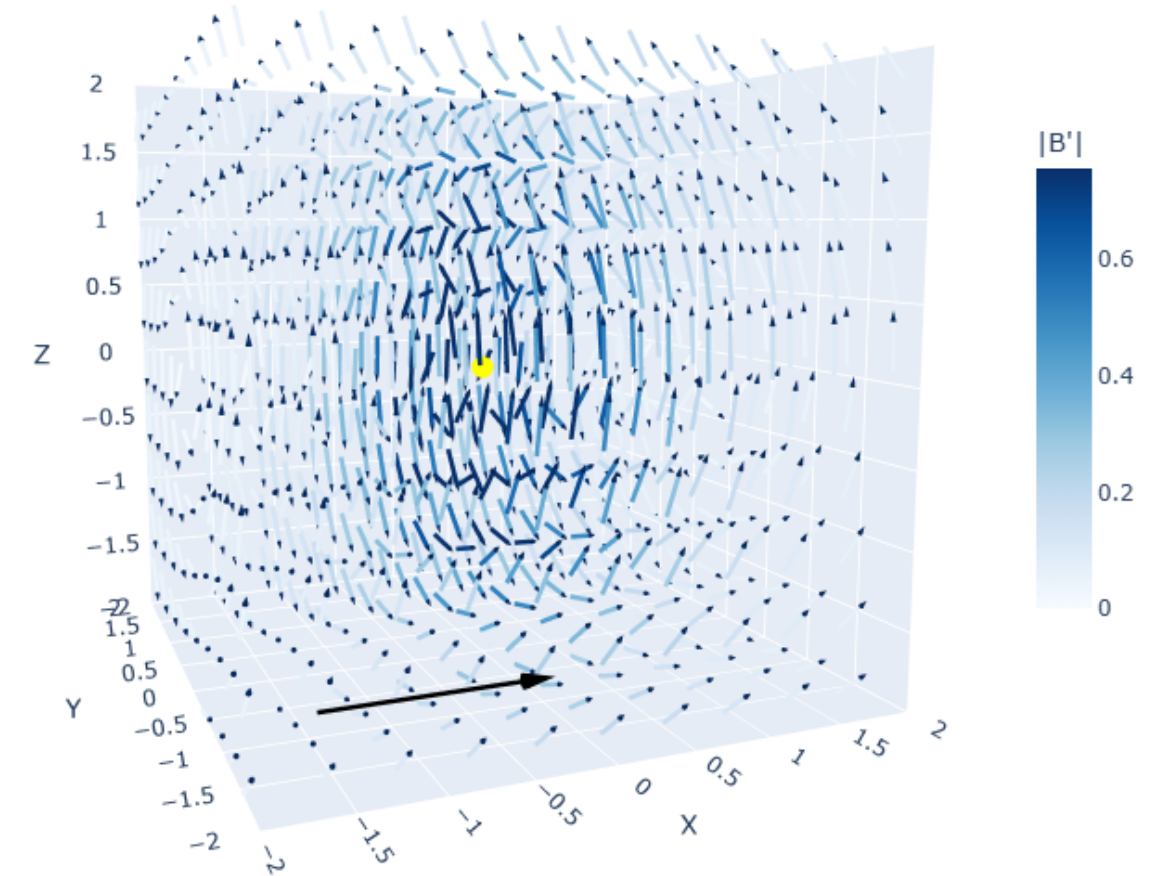


Relativistic EM Field

● Charges
— Velocity(v)

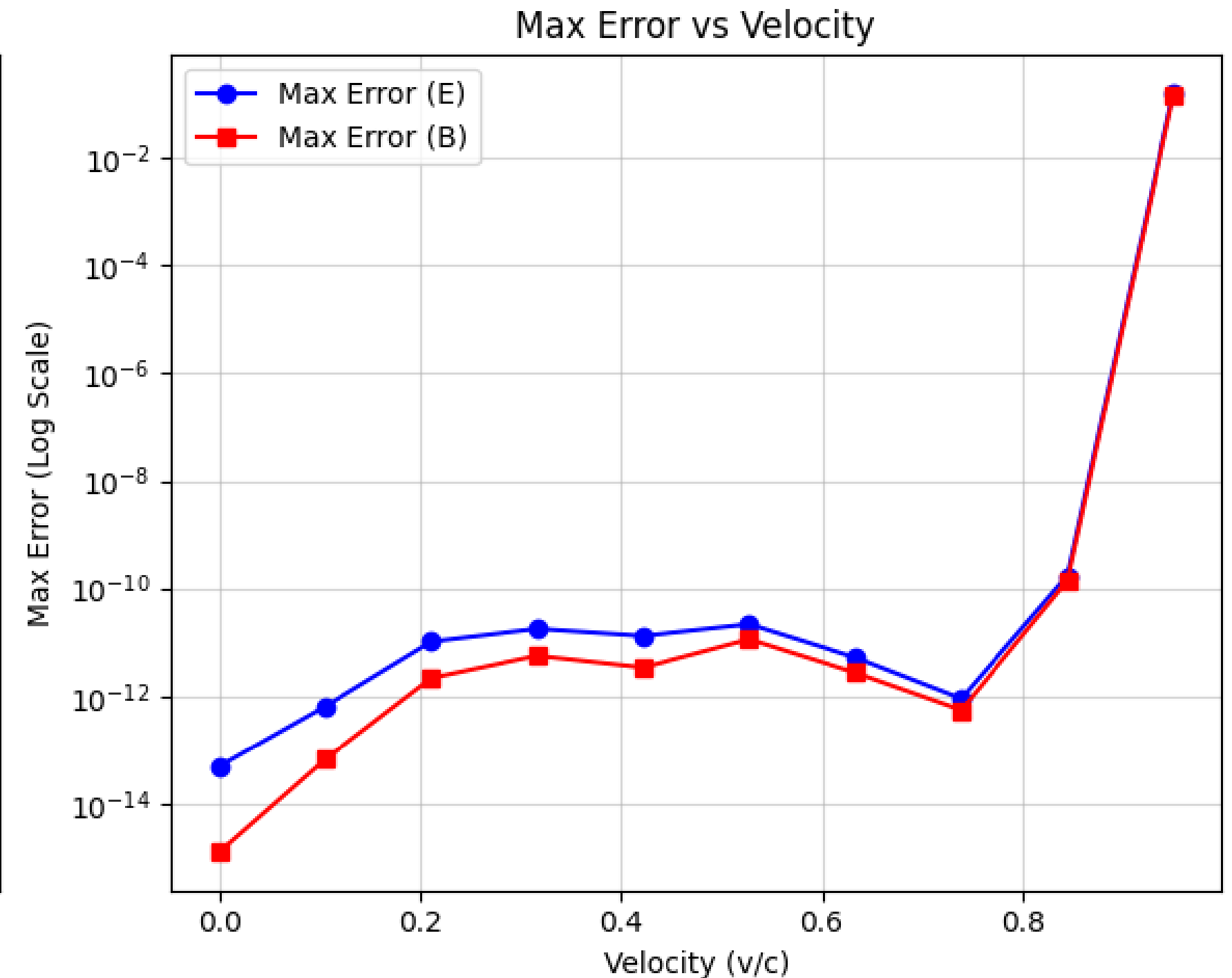
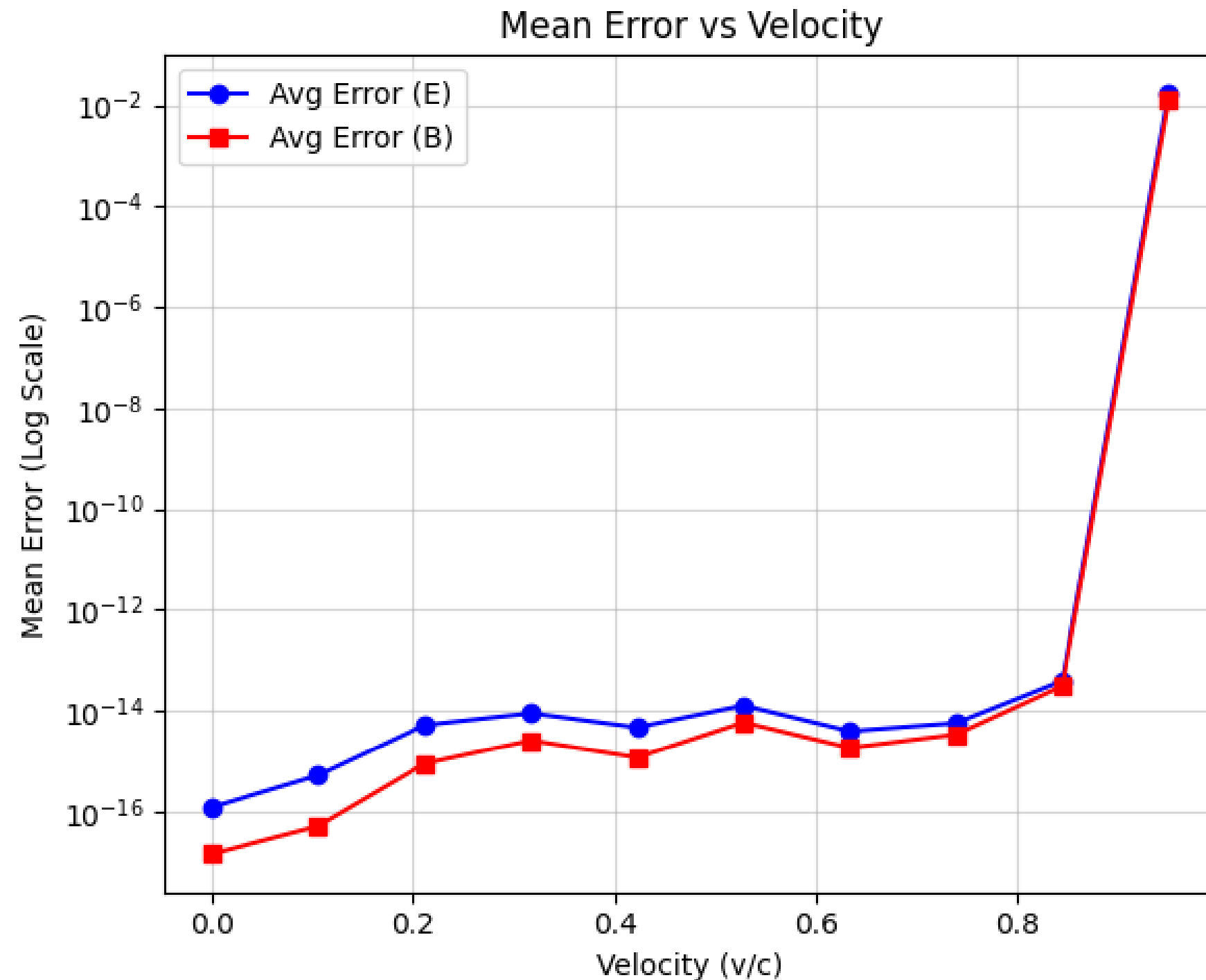


● Charges
— Velocity(v)



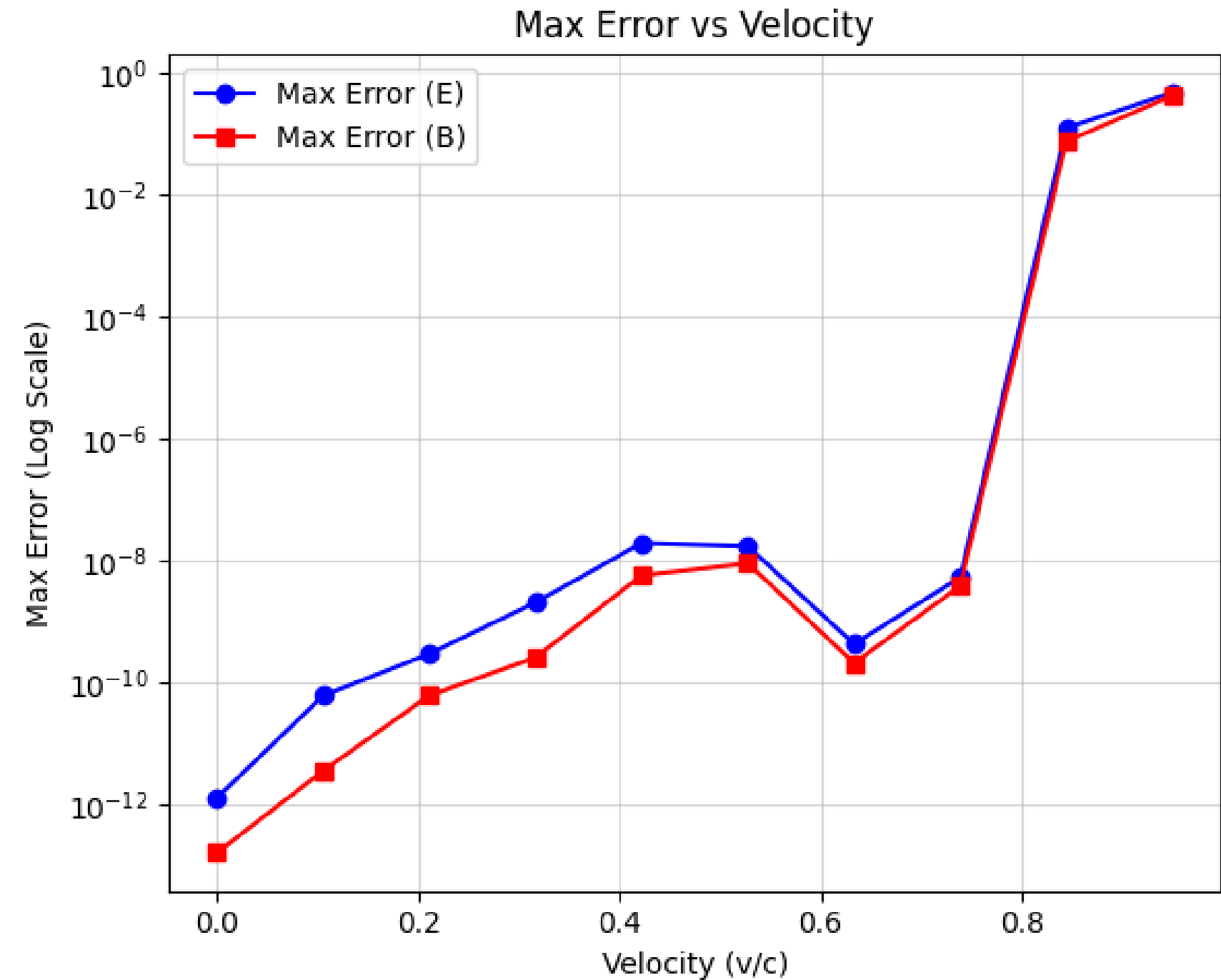
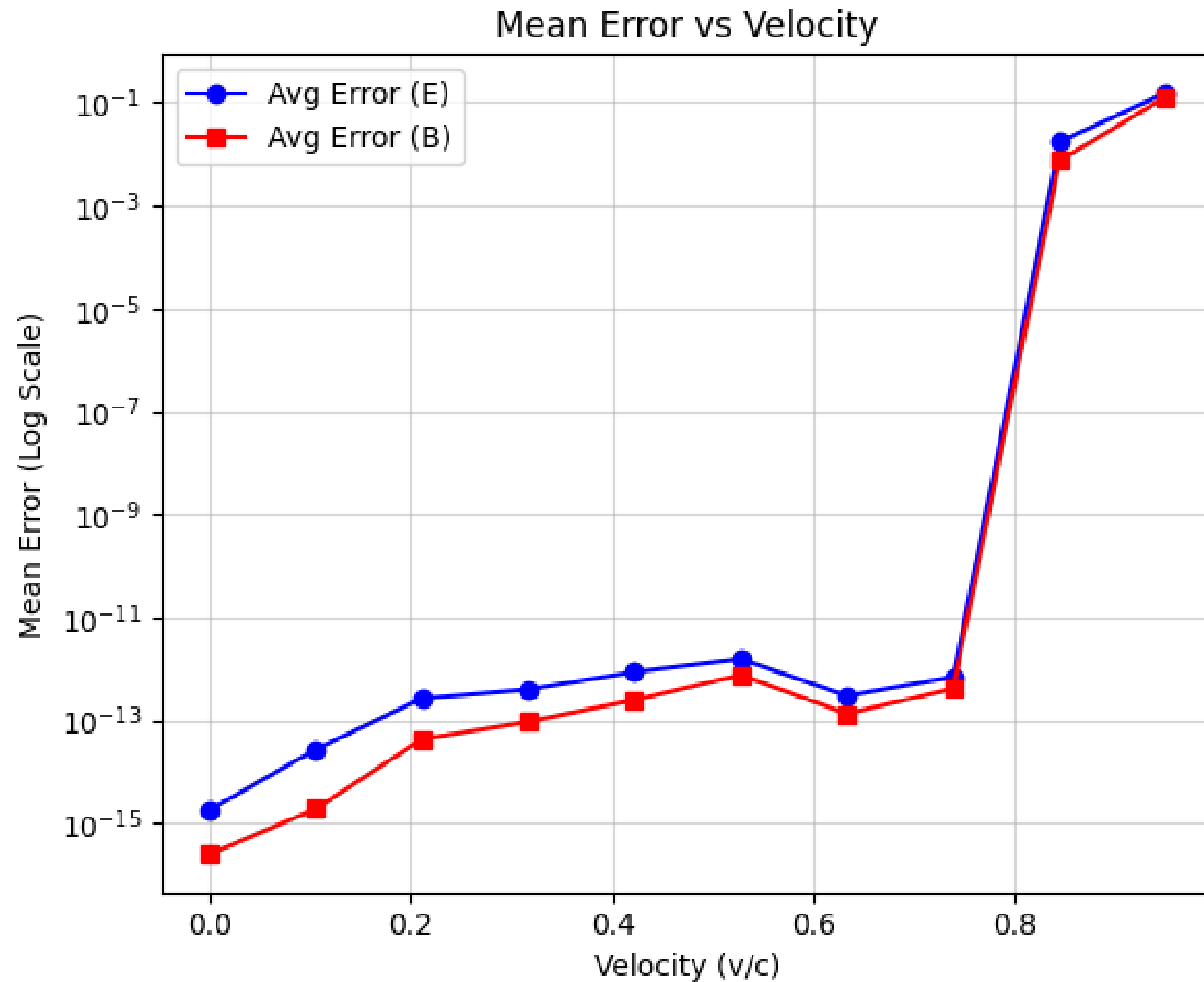
실행 결과-CovarianceVerifier

전하 1개($Q=+1$, $(0,0,0)$, Grid: $30 \times 30 \times 30$, $v=(v,0,0)$)



실행 결과-CovarianceVerifier

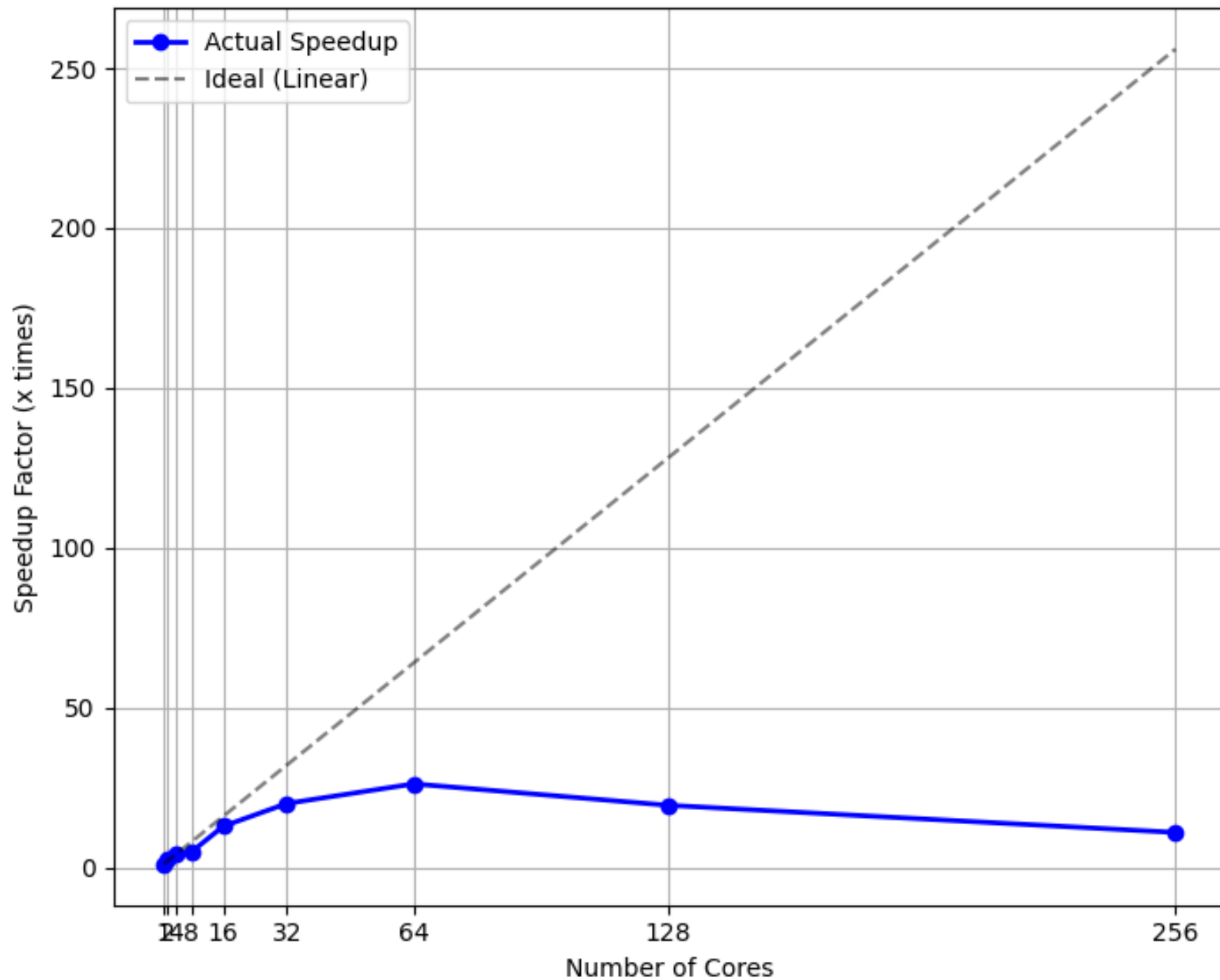
전하 100개(random 생성) Q range= $(-1,1)$, Grid: $30 \times 30 \times 30$, $v=(v,0,0)$



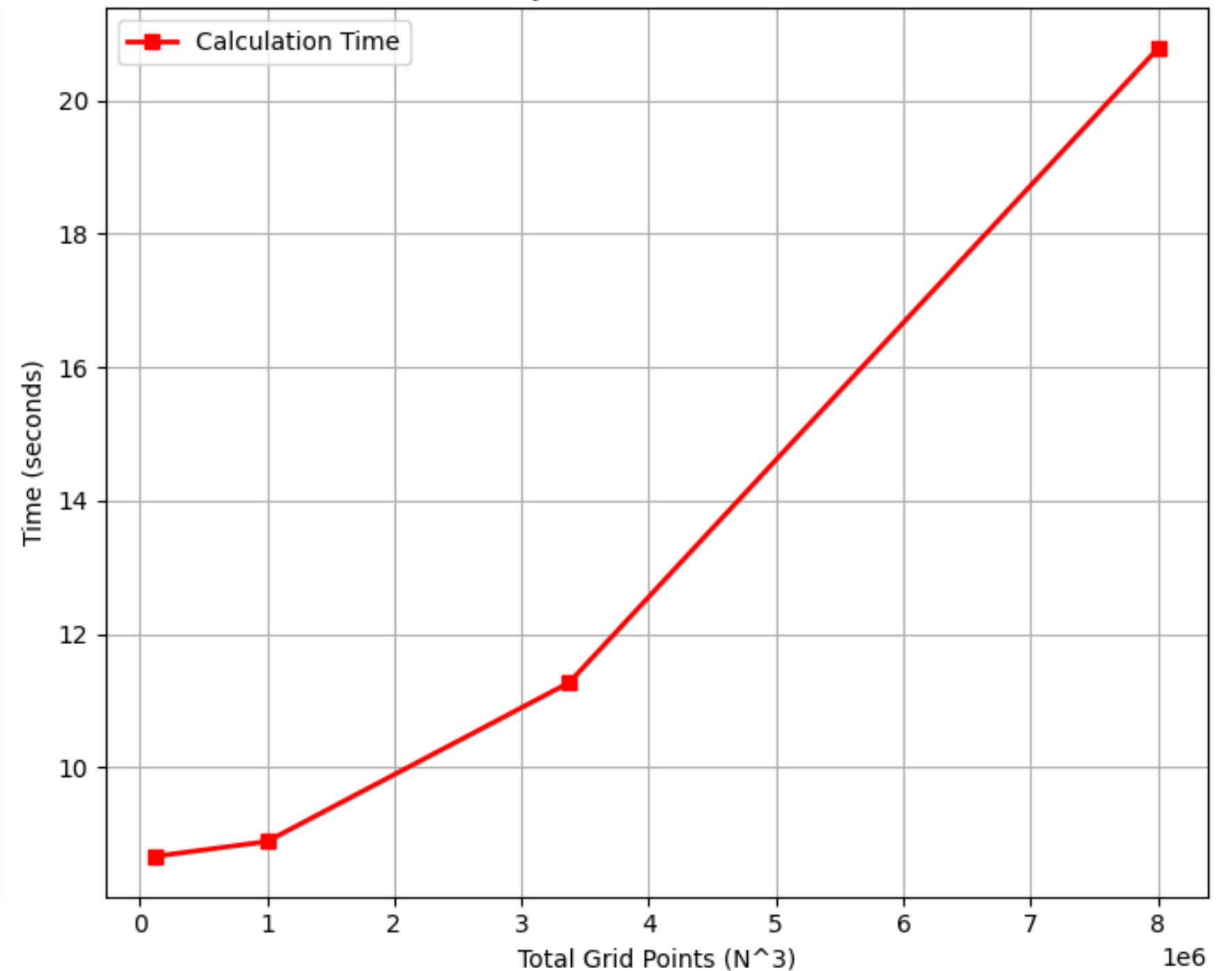
실행 결과-ParallelBenchmark

TensorField

Performance vs Cores (Amdahl's Law)



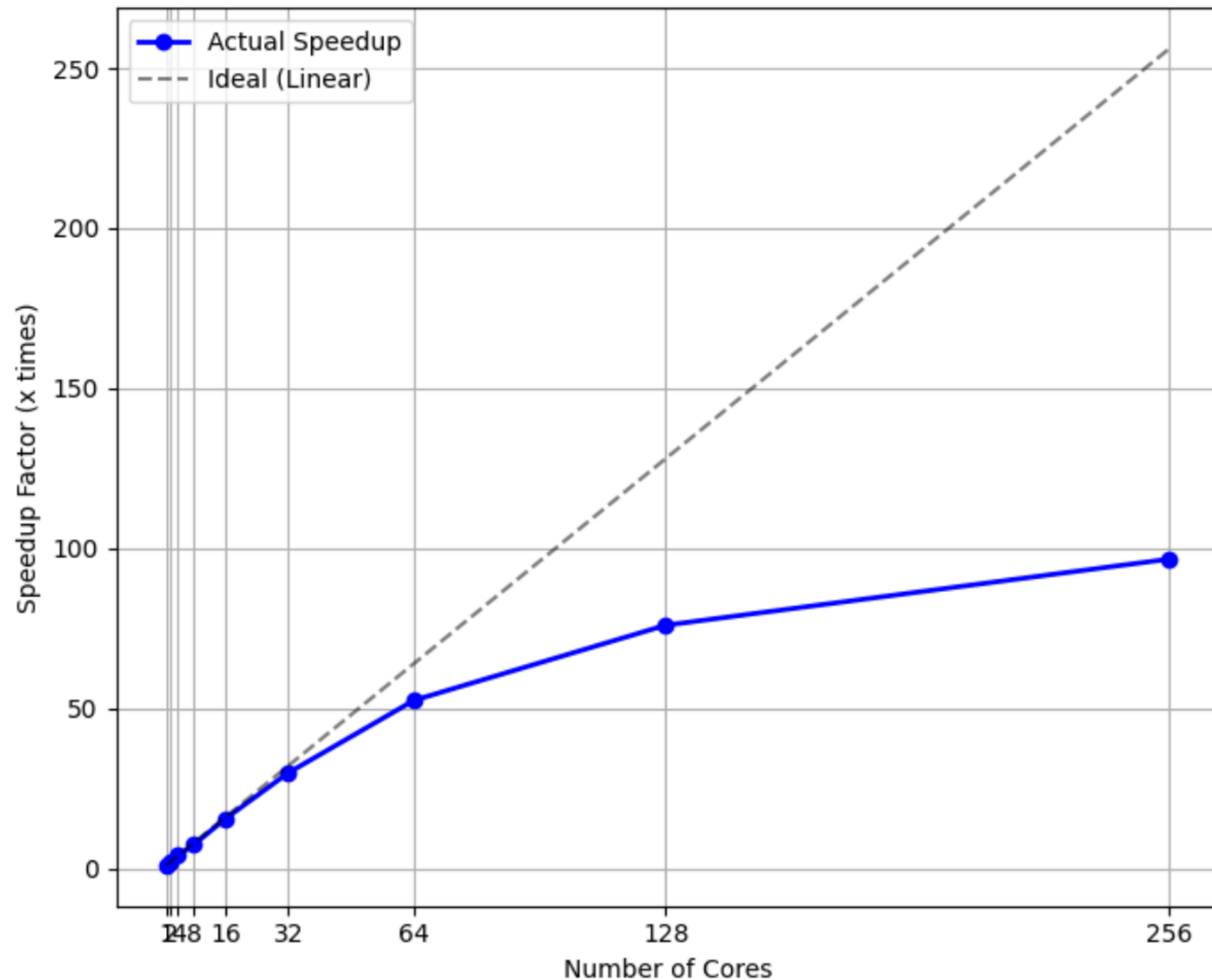
Scalability vs Grid Size (Max Cores)



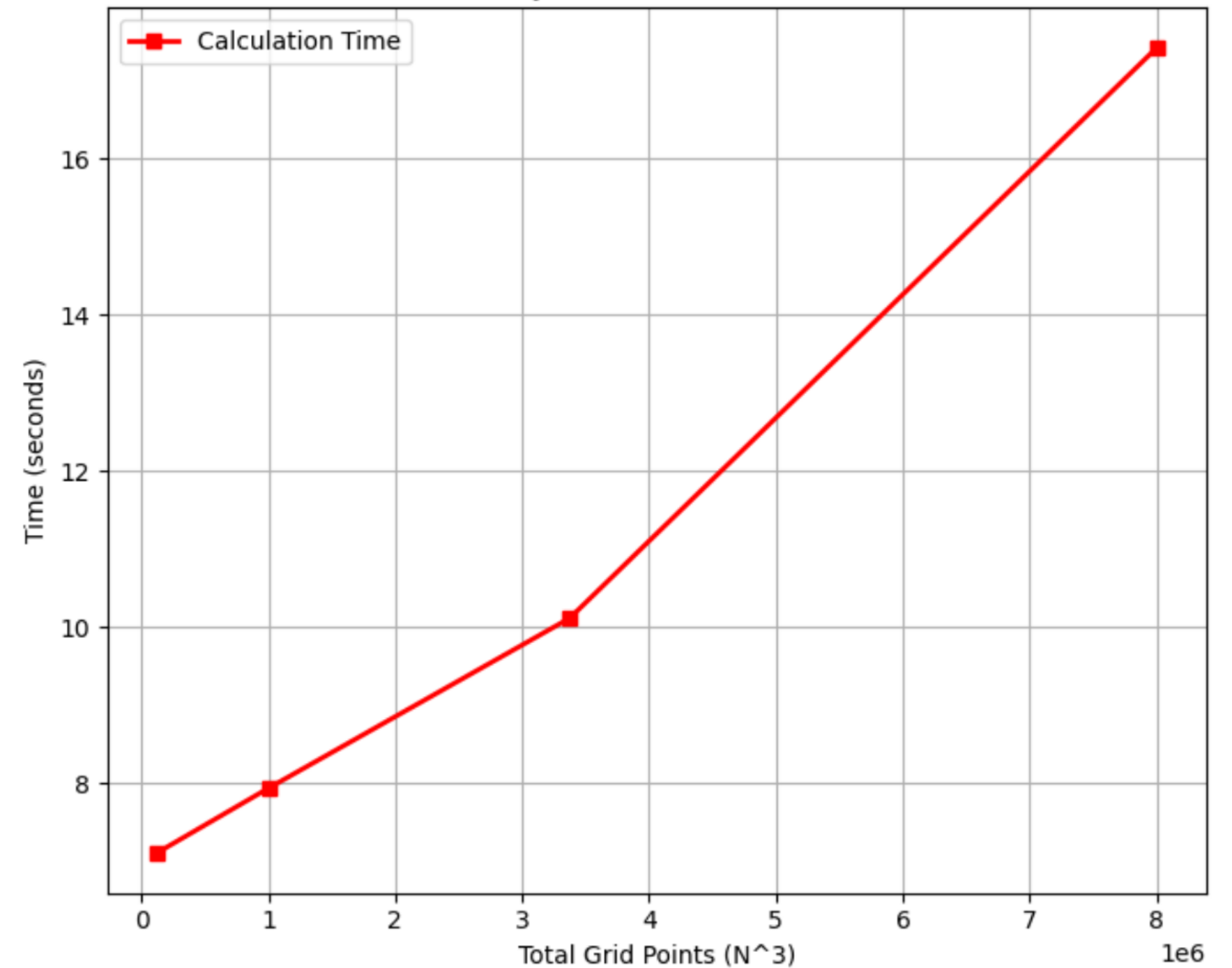
실행 결과-ParallelBenchmark

LWField

Performance vs Cores (Amdahl's Law)



Scalability vs Grid Size (Max Cores)



결론 CovarianceVerifier

- v 가 c 에 너무 가까워지지 않으면 작은 오차만 존재함.
- 이 때 오차는 float64의 부동소수점의 유효숫자인 15~16 자리 부근에서 발생함

```
tr=optimize.brentq(LWField._retarded_eq, t_min, t_max, args=(t,r0,q_r0,v0,a0),xtol=1e-12)
```

- 위 LWField 계산 코드 중에서, 가능 오차를 $xtol=1e-12$ 로 주어서 오차가 발생

```
t_max=t  
t_min=t-10*RANGE_LIMIT/c
```

$$\Delta t \approx \frac{R}{c-v} = \frac{R}{c(1-v/c)}$$

- 또한 c 에 가까워질때 오차가 큰 이유는 t 의 최솟값을 $t-10*(\text{관측하는 공간 한 방향의 길이}/2)/c$ 로 주었는데, 위의 식과 같이 찾아야하는 범위의 값이 발생하므로, v 가 c 에 가까워지면 관측해야하는 시간이 최솟값보다 작아지게 되어 값을 반환하지 못함. 이 지점을 그냥 continue로 넘어가서 오차 계산에 전부 포함됨.
- 전하의 개수가 많아지면, 각 지점마다 전하의 개수만큼의 계산을 하여, 오차가 누적되어 커짐.

결론 ParallelBenchmark

- TensorField는 코어가 많아질 때 병렬화의 효율이 좋지 못하는데, 이는 Tensor 계산은 병렬화하지 않고, 이를 계산하기 위한 static field를 구하는 함수만 병렬화했기 때문.
- 반면 LWField는 코어 수가 증가하며 효율이 감소하긴 했어도 전체 계산 자체를 병렬화 시켰기 때문에 어느정도 괜찮음.
- 두 필드 다 일정한 계산 크기 하에서는 격자 개수에 따라 거의 선형적으로 증가하는 시간을 보임.

계획

- LWField 클래스는 가속도를 포함한 일반적인 Liénard–Wiechert potential 공식을 사용함.
 - 이번엔 등속도 전하를 다루기 위해 가속도 항에 0을 입력했지만,
코드 구조 자체는 임의의 가속 운동에 대해서도 그대로 적용 가능함.
→ n개의 점전하에 대해 초기 조건을 주고 운동 방정식을 풀면서 시간에 의존해 전자기장을 업데이트하는 시뮬레이션으로 확장가능.
-